COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	8/4//2026	**DATE SUBMITTED:**	8/10//2026
NAME:	Denzen D. Igloso	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __5_

1. PROCEDURE OUTPUT

ILO A Code	Analysis
	This is the declaration of the function that are going to be used.
	This part of code is in the driver function and it only declare the variable of the queue which is the “q” and the next lines are pushing the c, p, e, 0, 1, 0.

```
    // display
    display(q);
    // empty
    std::cout << " "<< q.empty() << "\n";
    // size
    std::cout << "The size of the queue are: "<< q.size() << "\n";
    // front
    std::cout << "The front of the queue: "<< q.front() << "\n";
    // back
    std::cout << "The back of the queue: "<< q.back() << "\n";
    // pop
    std::cout << "Popping the front: " << q.front() << "\n";
    q.pop();
    std::cout << "After popping: ";
    // display
    display(q);
```

In this part of code, first one is the display function that display the queue and the next is size function where it gives us the size of the queue, the next is the front and back which tells us the front and back of the queue, the next one is the pop which delete the first in the queue, and last is display it againg.

```
    //push another element
    std::cout << "\nPushing an element: ";
    q.push('D');
    display(q);
    std::cout << "\nPushing an element: ";
    q.push('S');
    display(q);
    std::cout << "\nPushing an element: ";
    q.push('A');
    display(q);
    deleteQ(q);
}
```

This part of code where i add another elements into the queue, i use the push function where it adds automatically at the back of the queue and also i display it.

```
void display(std::queue<char> r){
    // create a copy of queue
    std::queue<char> c = r;
    while(!c.empty()){
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << "\n";
}

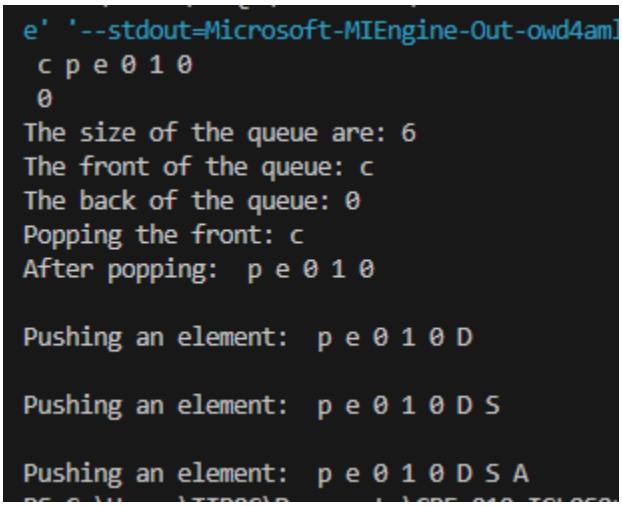
```

This is the void display function where it creates a copy of the queue and that new queue checks the front and display it then pop it through out the queue because it is in the while loop

```
void deleteQ(std::queue<char> r){
    std::queue<char> c = r;
    while(!c.empty()){
        c.pop();
    }

```

This is the void deleteQ where it creates also a new copy of queue and then delete it.

	This is the output of the full code. It display the function that i used.

ILO B CODE	Analysis

HOA5p1 > ILO b > queueII.h > Display<T>(std::queue<char>)
#ifndef QUEUELL_LL
#define QUEUELL_LL
 	
template <typename T>
class qNode{
public:
T data;
qNode* next;
};
 	
// creating a new node
template <typename T>
qNode<T>* new_node(T newData){
qNode<T>* newNode = new qNode<T>;
newNode->data = newData;
newNode->next = nullptr;
return newNode;
}
 	
template <typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
// create a new node
qNode<T>* newNode = new_node(newData);
 	
// inserting to an empty queue
if((*frontPtr) == nullptr && (*backPtr) == nullptr){
(*frontPtr) = newNode;
(*backPtr) = newNode;
	This part show only the declaration of the functions enqueue, dequeue, and creating the new node.

```
        (*backPtr) = newNode;
    }
    // inserting an item into a non empty queue
    //point the backPtr next to the newNode
    (*backPtr)->next = newNode;
    (*backPtr)= newNode;
}

template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);
    // check if the queue is only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
    // deleting of the node
    (*frontPtr) = deleteNode->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}
```

```
}
// display all elements in the list
template <typename T>
void Display(std::queue<char> r){
    std::queue<char> c = r;
    while(!c.empty()){

        c.pop();
    }
    std::cout << "\n";
}
// return the front
template <typename T>
void f_ront(qNode<T>** frontPtr){
    if((*frontPtr) != nullptr){
        std::cout << (*frontPtr)->data << std::endl;
        return;
    }
}
#endif
```

This part is where i create a display and return front, but the display function is not done so its not working while the return front is working

main.cpp ...\ILO A | main.cpp ...\ILO b 1

HOA5p1 > ILO b > main.cpp > main()

```
#include<iostream>
#include"queue11.h"

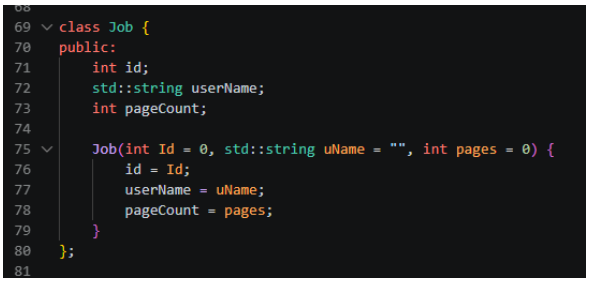
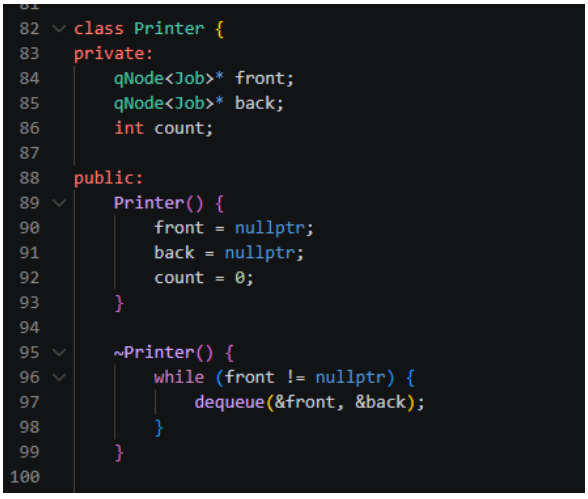
int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue(&front, &back, 'D');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'E');
    std::cout << front->data << " " << back->data << std::endl;
    enqueue(&front, &back, 'N');
    std::cout << front->data << " " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data << std::endl;
    f_ront(&front);
}
```

This is the main function where the function from the header file was used

	This is the output of the code where it prints the front and back of the queue and the last line of the output is the return front of the queue.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Printer Simulation Header file	Observation
	This will print only print the queue
	This creates the Job class that stores the id, user name, and page count for each print job.
	This creates the Printer class, initializes the queue with null pointers and 0 count, and deletes all leftover jobs when the program ends.

```cpp
template <typename T>
void printQueue(qNode<T>* frontPtr) {
    if (frontPtr == nullptr) {
        std::cout << "Queue is empty\n";
        return;
    }

    qNode<T>* current = frontPtr;
    int pos = 1;
    while (current != nullptr) {
        std::cout << " " << pos++ << ". Job ID: " << current->data.id
                  << " | User: " << current->data.userName
                  << " | Pages: " << current->data.pageCount << "\n";
        current = current->next;
    }
}
```

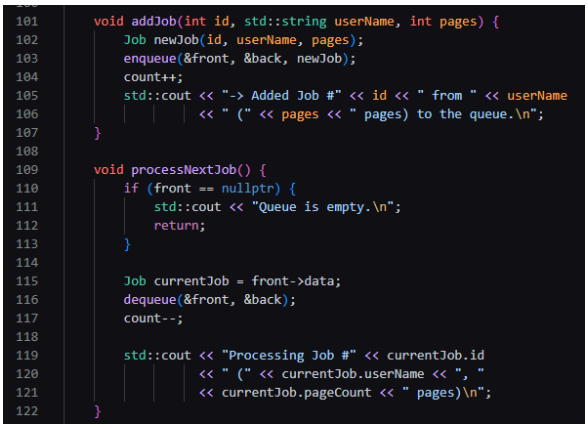
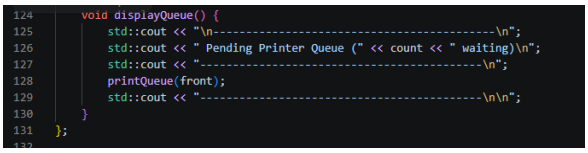
```cpp
class Job {
public:
    int id;
    std::string userName;
    int pageCount;

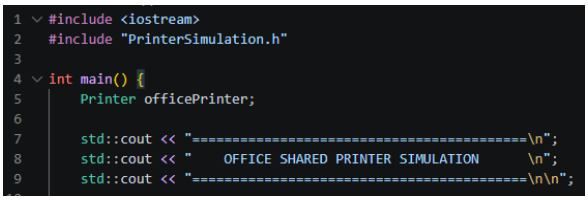
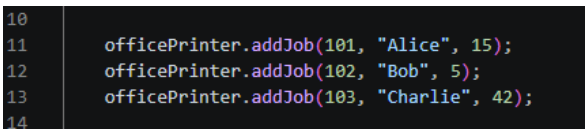
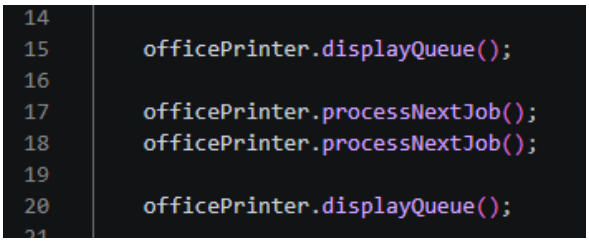
    Job(int Id = 0, std::string uName = "", int pages = 0) {
        id = Id;
        userName = uName;
        pageCount = pages;
    }
};
```

```cpp
class Printer {
private:
    qNode<Job>* front;
    qNode<Job>* back;
    int count;

public:
    Printer() {
        front = nullptr;
        back = nullptr;
        count = 0;
    }

    ~Printer() {
        while (front != nullptr) {
            dequeue(&front, &back);
        }
    }
```

	The addJob will adds a new print to the back of the queue, updates the job count, and displays a confirmation message. And the processNextJob will checks if there are jobs in line, processes and removes the front job, and decreases the total count.
	This prints the header lines and total number of waiting jobs, then calls printQueue to display all the pending jobs in order.

Main cpp	Observation
	This code declared the Printer function from the header file as the officePrinter. And it prints the Title on the output.
	This will take the documents, id, users name, and the number of pages, and form the queues from different pcs or users
	It displays the queues and for the processNextJob(), it will process the first in line and prints it and the line on 20 will display it again

```
21
22      officePrinter.addJob(104, "Diana", 8);
23      officePrinter.addJob(105, "Evan", 12);
24
25      officePrinter.displayQueue();
26
27      officePrinter.processNextJob();
28      officePrinter.processNextJob();
29      officePrinter.processNextJob();
30
31      officePrinter.displayQueue();
32
33      return 0;
34  }
```

This is the same from above, it will again add that 2 documents from different users and display the queue. Then process it or prints it if you call the processNextJob()

OUTPUT

```
========================================
    OFFICE SHARED PRINTER SIMULATION
========================================

-> Added Job #101 from Alice (15 pages) to the queue.
-> Added Job #102 from Bob (5 pages) to the queue.
-> Added Job #103 from Charlie (42 pages) to the queue.

------------------------------------------
 Pending Printer Queue (3 waiting)
------------------------------------------
 1. Job ID: 101 | User: Alice | Pages: 15
 2. Job ID: 102 | User: Bob | Pages: 5
 3. Job ID: 103 | User: Charlie | Pages: 42
------------------------------------------

Processing Job #101 (Alice, 15 pages)
Processing Job #102 (Bob, 5 pages)

------------------------------------------
 Pending Printer Queue (1 waiting)
------------------------------------------
 1. Job ID: 103 | User: Charlie | Pages: 42
------------------------------------------

-> Added Job #104 from Diana (8 pages) to the queue.
-> Added Job #105 from Evan (12 pages) to the queue.
```

```
----------------------------------------
 Pending Printer Queue (3 waiting)
----------------------------------------
 1. Job ID: 103 | User: Charlie | Pages: 42
 2. Job ID: 104 | User: Diana | Pages: 8
 3. Job ID: 105 | User: Evan | Pages: 12
----------------------------------------

Processing Job #103 (Charlie, 42 pages)
Processing Job #104 (Diana, 8 pages)
Processing Job #105 (Evan, 12 pages)

----------------------------------------
 Pending Printer Queue (0 waiting)
----------------------------------------
Queue is empty
----------------------------------------
```

Defend your choice of internal representation: Why did you use arrays or linked lists?

- I selected a linked list data structure for the printer queue as opposed to an array since a printer queue changes in size all the time as new print jobs come in and as old ones are completed. Using an array structure, I would be forced to give an upper bound size, which means every time a print job at the front is finished, every other item would have to be moved to make up for the deletion. On the other hand, using a linked list with both front and rear pointers, I am able to append new items at the end and remove items from the beginning without having to rearrange any other items.

Conclusion

- During this activity, I understood how FIFO queues can be implemented with the help of linked lists using custom Job and Printer classes. The whole process was very organized beginning from simple node operations to higher level operations such as addJob() and processNextJob(). It was a very good practical approach to understand how printers work in offices in a proper manner without losing any data. All in all, I believe I performed well since my simulation worked fine and memory allocation was done properly, even though the one thing that could use some improvement is to check variables in other functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google, gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **I used google and gemini for simply to understand the concepts of the code and grammar correction, i also use it to help me understand when i got stuck on the same code for almost 3 hrs.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.